

## Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

### Übung Nr. 2 Abgabe KW 18

#### 1. Aufgabe (Codeopolis)

In der Klasse `Game` werden die Spielrunden in einer Schleife ausgeführt, bis das Spiel beendet ist. Implementieren Sie dies rekursiv.

#### 2. Aufgabe

Implementieren Sie ein Programm, das alle Permutationen der Elemente eines `int[]`-Arrays ausgibt. Nutzen Sie dabei keine explizite Schleife sondern verwenden Sie einen rekursiven Ansatz.

#### 3. Aufgabe

Palindrome sind Worte oder allgemein Zeichenketten, die von links und von rechts gelesen identisch sind. Beispiel sind Otto, Reittier oder Rentner.

- (a) Implementieren Sie ein Programm, welches für einen gegebenen Eingabestring feststellt, ob dieser ein Palindrom ist oder nicht. Zur Eingabe des zu untersuchenden Strings soll ihr Programm zwei Möglichkeiten bieten. Zum einen soll der Eingabestring beim Programmaufruf als Argument übergeben werden können. Zum anderen soll es aber auch möglich sein, den Eingabestring aus einer Textdatei zu lesen.

Das Problem kann sowohl iterativ als auch rekursiv gelöst werden. Implementieren Sie beide Lösungen. Entwerfen Sie ihr Programm so, dass es leicht um weitere Algorithmen zur Lösung des Problems erweiterbar ist.

Beachten Sie unbedingt folgende Implementierungsdetails:

- Erzeugen Sie ein Interface `Palindrom` mit der Methode `boolean istPalindrom(String wort)`.
  - Nennen Sie die erste Implementierung `PalindromRekursiv`.
  - Nennen Sie die zweite Implementierung `PalindromIterativ`.
- (b) Implementieren Sie ein Programm, welches zwei Messreihen (iterative und rekursive Implementierung) erzeugt und in eine Datei schreibt. Es soll die Geschwindigkeit der Berechnung für verschiedene Problemgrößen (Länge der Strings bzw. Wortlänge) gemessen und als Comma-separated values in eine Textdatei geschrieben werden.
- (c) Führen Sie Messungen für ihre iterative und ihre rekursive Lösung durch und tragen Sie beide Messreihen in einem Diagramm gegeneinander ab. Zum Beispiel so wie in Abbildung 1 dargestellt. Entscheiden Sie selbst, bis zu welcher Problemgröße Sie Messungen durchführen. Bei der Präsentation ihrer Lösung sollten Sie in der Lage sein, die gemessenen Effekte zu erklären.

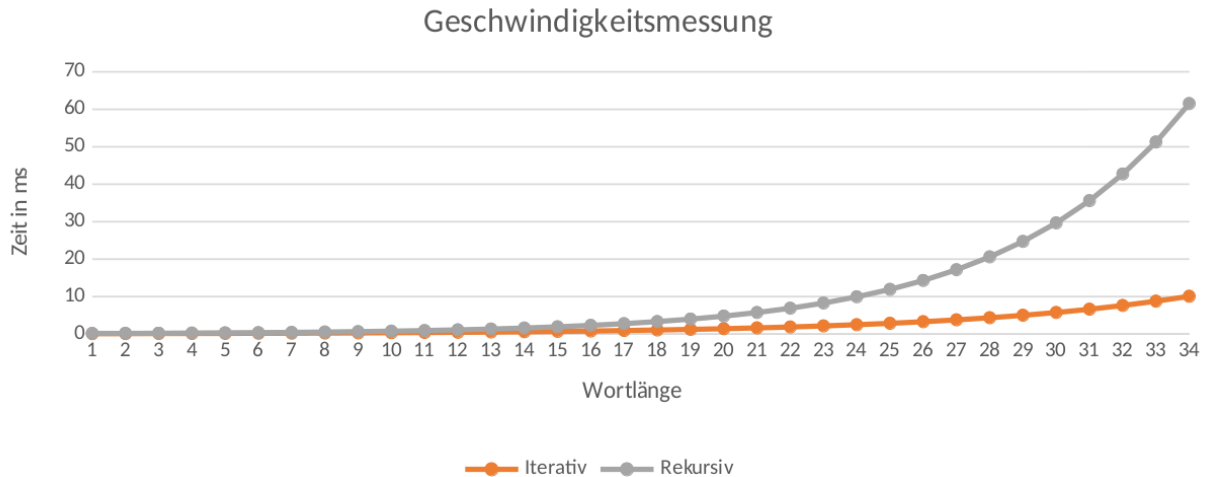


Abbildung 1: Beispiel für eine Messreihe zum Vergleich der iterativen und der rekursiven Implementierung.

#### Hinweise:

- Bei der Geschwindigkeitsmessung kann die Klasse `System.nanoTime()` hilfreich sein.
- Zur Erzeugung des Diagramms in Teilaufgabe c kann ein beliebiges Tabellenkalkulationsprogramm verwendet werden. Eine andere Möglichkeit, die jedoch mehr Einarbeitungsaufwand erfordert, ist beispielsweise die Nutzung von Gnuplot.

#### 4. Aufgabe

Das klassische Türme-von-Hanoi-Problem besteht darin,  $n$  unterschiedlich große Scheiben von einem *Startstab* (Quelle) auf einen *Zielstab* zu bewegen. Abbildung 2 illustriert die Startanordnung der Türme von Hanoi. Sie haben dabei drei Stäbe (oder Türme) zur Verfügung:

- Einen Startstab (Quelle), auf dem anfangs alle Scheiben übereinandergestapelt sind (größte Scheibe unten, kleinste oben).
- Einen Zielstab, auf den alle Scheiben im Verlauf des Algorithmus bewegt werden sollen.
- Einen Hilfsstab, der während des Verschiebens der Scheiben verwendet werden kann.

Die Regeln lauten:

1. Es darf immer nur **eine** Scheibe bewegt werden.
2. Keine größere Scheibe darf auf einer kleineren Scheibe abgelegt werden.

Folgender Algorithmus löst die Türme von Hanoi rekursiv:

```
function bewege (Zahl i, Stab a, Stab b, Stab c) {
    falls (i > 0) {
        bewege(i-1, a, c, b);
        verschiebe oberste Scheibe von a nach c;
        bewege(i-1, b, a, c);
    }
}
```

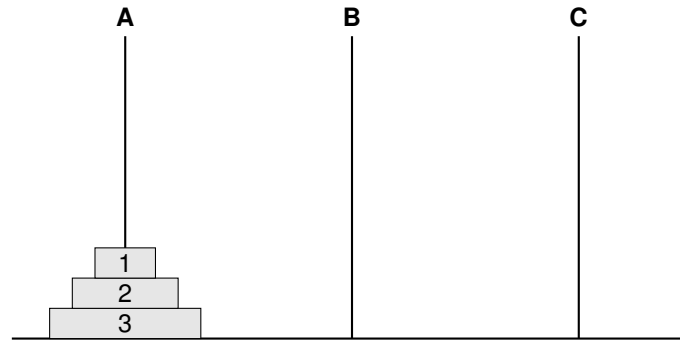


Abbildung 2: Einfache Darstellung der Türme von Hanoi mit drei Scheiben.

Implementieren Sie eine rekursive Lösung für die Türme von Hanoi und erklären Sie im Review, warum der Algorithmus das Problem löst.

### Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

- ... den Unterschied zwischen iterativer und rekursiver Implementierung erklären können.
- ... in der Lage sein, rekursive Methoden zu implementieren.
- ... eine iterative Implementierung in eine rekursive Implementierung umwandeln können.

## Zusatzaufgaben

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

### 1. Aufgabe (Codeopolis)

Implementieren Sie die Methode `Dialog.readPositivIntegerInput()` rekursiv.

### 2. Aufgabe

Mithilfe des Algorithmus von Euklid kann der größte gemeinsame Teiler (ggT) zweier Zahlen sehr einfach rekursiv berechnet werden. Nach Euklid gilt für zwei natürliche Zahlen  $a$  und  $b$  folgendes:

$$\begin{aligned} ggT(a, 0) &= a \\ ggT(a, b) &= ggT(b, a \bmod b) \end{aligned}$$

Implementieren Sie ein Programm, welches den ggT für zwei als Argumente übergebene natürliche Zahlen sowohl rekursiv als auch iterativ berechnet.

### 3. Aufgabe

Schreiben Sie ein Programm, das den Inhalt eines gesamten Verzeichnisses rekursiv durchsucht und dabei den Pfad jeder gefundenen Datei ausgibt. Dabei sollen auch alle Unterverzeichnisse erfasst werden.

Um auf das Dateisystem zuzugreifen und Dateien/Verzeichnisse zu verarbeiten können Sie entweder die API `java.io.File`<sup>1</sup> oder `java.nio.file`<sup>2</sup> nutzen.

## Erweiterungen

- Lassen Sie das Programm nur bestimmte Dateitypen anzeigen (z. B. `.txt`).
- Erfassen Sie, wie viele Dateien Sie insgesamt durchsucht haben, und geben Sie am Ende eine Zusammenfassung aus.
- Zählen Sie, wieviele Dateien Sie von den unterschiedlichen Dateitypen erfasst haben.

<sup>1</sup><https://docs.oracle.com/javase/8/docs/api/java/io/File.html>

<sup>2</sup><https://docs.oracle.com/javase/8/docs/api/java/nio/file/package-summary.html>